

1. Single Inheritance

```
class Animal {  
    void eat() {  
        System.out.println("Eating");  
    }  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat();  
        d.bark();  
    }  
}
```

2. Multiple Inheritance

```
interface A {  
    void show();  
}
```

```
interface B {  
    void display();  
}  
  
class C implements A, B {  
    public void show() {  
        System.out.println("Show Method");  
    }  
  
    public void display() {  
        System.out.println("Display Method");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.show();  
        obj.display();  
    }  
}
```

3. Multilevel Inheritance

```
class Animal {  
    void eat() {  
        System.out.println("Eating");  
    }  
}
```

```
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking");  
    }  
}
```

```
class Puppy extends Dog {  
    void weep() {  
        System.out.println("Weeping");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Puppy p = new Puppy();  
        p.eat();  
        p.bark();  
        p.weep();  
    }  
}
```

4. Hierarchical Inheritance

```
class Animal {
```

```
void eat() {  
    System.out.println("Eating");  
}  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking");  
    }  
}
```

```
class Cat extends Animal {  
    void meow() {  
        System.out.println("Meowing");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        Cat c = new Cat();  
  
        d.eat();  
        d.bark();  
  
        c.eat();  
    }  
}
```

```
        c.meow();  
    }  
}
```

5. Hybrid Inheritance

```
interface Animal {  
    void eat();  
}
```

```
class Dog {  
    void bark() {  
        System.out.println("Barking");  
    }  
}
```

```
class Puppy extends Dog implements Animal {  
    public void eat() {  
        System.out.println("Eating");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Puppy p = new Puppy();  
        p.eat();  
        p.bark();  
    }  
}
```

```
}  
}
```

Constructors

1. Default Constructor

```
class Student {  
    Student() {  
        System.out.println("Default Constructor");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
    }  
}
```

2. Parameterized Constructor

```
class Student {  
    Student(String name) {  
        System.out.println("Name: " + name);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {
```

```
Student s = new Student("Prasanna");  
}  
}
```